A2 FILE CMD

The complete manual

Two panels. One Apple II.

6502 and 65C02 editions
BOOT · EXTRA · XL

By Arnaud Verhille

Printable edition of the project's English manual.
Source: docs/MANUAL.md

Generated 9 September 2026. This PDF reproduces the documentation available when it was generated; development may continue in the source manual.

Contents

The A2 File Cmd manual

A two-pane ProDOS file manager in the spirit of Total Commander, for the 128 KB Apple IIe; on screen it calls itself **A2 FILE CMD 0.7.5** (the number lives in `A2FC_VERSION` in the Makefile, picked up by the launcher, the status line and the help). It is free software under the GNU GPL v3, by Arnaud Verhille; the launcher and the help say so. Two ways to start it: boot the `/A2FC6502` or `/A2FC65C02` floppy, or pick `A2FILE.SYSTEM` from a selector such as Bitsy Bye.

The launcher first checks the machine in plain 6502 code. Both CPU builds require 128 KB and an 80-column card. The 65C02 build additionally rejects an unenhanced IIe before executing enhanced instructions; the 6502 build accepts it. A failed check displays a 40-column message and a key returns to ProDOS. `bench/machine.py` tests the insufficient-memory refusal.

The splash screen identifies both the CPU and edition, for example `6502 FLOPPY EDITION` on BOOT or `65C02 COMPLETE EDITION` on XL. It shows the machine requirements, available tools, the date and time if a clock is present, then **PLEASE WAIT**. Then come the two panels in 80-column text: on the left the boot volume, on the right its `DEMO` directory the first time (the `.2mg` has one; the bare floppy has not, and the right panel then shows the list of volumes); afterwards the two directories, the sort order and the active panel from the previous session, read from `A2FILE/A2FILE.CFG`.

Each panel lists a directory: name, ProDOS type, auxtype and size in bytes, directories first (with their block count), then the files, sorted by name, size or type according to the chosen mode (the star in the header says which). The inverse line is the selection; the path of the active panel is inverse too. The separator line carries the free space of the active panel's volume. Line 22 details the selected entry (type, auxtype, blocks, bytes, modification date) and the number of tagged files, line 23 receives the messages and the questions, and the last line is the key bar, Norton Commander style: each key in an inverse block, its label in plain text right after. The viewers have their own, with the path and the page on the left.

Keys

Key	Action
Up / Down	move the selection
Left / Right (or **< / >**, **- / +**)	previous / next page (18 lines): this is the fast move through a long directory, the Apple IIe keyboard having no PgUp
[/]	first / last entry
TAB	switch panel
=	open the active panel's directory in the other panel
SPACE	tag or untag the selected file (star after the name); the cursor stays put
*	invert the panel's tags
' then a key	jump to the next entry whose name starts with that letter or digit, as in Bitsy Bye
RETURN	open: a directory opens; a disk image `.PO`/`.DSK`/`.2MG`, ProDOS or DOS 3.3, opens read-only as a folder (see "A disk image as a folder"); a picture shows full screen, in HGR or DHGR depending on its content (a key to return, the message line names the recognized format); a TXT is read page by page; a SYS or a BAS runs after confirmation; any other file shows in hexadecimal

Key	Action
ESC	up to the parent directory, the selection returning to the directory just left; from a volume root, the list of volumes
/	the list of on-line volumes
C	copy the tagged entries, otherwise the selected one, into the other panel's directory; a directory is copied whole, subdirectories included, an already-present subdirectory is filled in; same name, same type and auxtype. When the file exists, A2FC asks: **O** overwrite, **S** skip, **A** overwrite all, **N** overwrite none. A progress bar shows the current file, its rank out of the total and the bytes copied; the final message counts the files copied and skipped
V	move: copy, then delete the original, directories included
R	rename (ProDOS name: a letter, then letters, digits or periods, 15 at most)
D	delete the tagged entries, otherwise the selected one, after a confirmation; a directory is deleted with all its contents
K	make a directory in the active panel
S	change the sort: name, decreasing size, type; both panels follow
M	mark the files absent from the other panel, or of a different size or modification date: followed by C, it is a synchronization
A	change a file's type and auxtype, in hexadecimal
L	lock or unlock; a locked file carries an L after its name and refuses deletion and renaming
?	the help, a screen that sums up every key, under the title "A2 File Cmd"
T	read the selected file as text; an Applesoft (`BAS`) program is listed detokenized (line numbers and keywords) instead of hex, by the `BASLIST.PLG` overlay; an AppleWorks (`AWP`) document is rendered by `AWP.PLG`
H	show the selected file in hexadecimal
X	run the selected file after confirmation; A2FC does not come back. A SYS is read at `$2000`, a BIN at its auxtype, between `$0800` and `$BAFF` (the stub keeps its ProDOS buffer at `$BB00`). A BAS (Applesoft) goes through `BASIC.SYSTEM`, see below
E	edit the selected file as text; on a directory or `..`, create a new text file in the current directory
I	show the selected file as a picture, whatever its name: HGR or DHGR, raw or RLE-compressed
P	pause or resume the Mockingboard music; RETURN on a `.MB` file starts it
W	disk images: write a `.PO`/`.DSK`/`.2MG` to a floppy, read a floppy into a fresh image, copy one floppy onto another (see below)
!	the overlay menu: the list of `A2FILE/*.PLG` with their description, each run on the selection (Up/Down one line, Left/Right a page, a letter jumps to the next name starting with it). Among them: **COMPARE** confronts the selection with the file of the same name in the other panel byte by byte, and **SEARCH** asks for a text and tags the panel files that contain it (case-insensitive)

Key	Action
F	open the `FORMAT.PLG` overlay; Escape returns directly to the panels
1 … **0**	the ten buttons of the key bar, in order, like Norton Commander and A2Command
Ctrl-T / **Ctrl-N**	tag all / untag all; **Ctrl-R** re-reads both panels (floppy swapped, `/RAM` rebuilt)
Q	quit to ProDOS after confirmation: Bitsy Bye takes over

In a picture, **Left** and **Right** move to the previous or next picture in the same directory without returning to the panels: the DHGR directory is paged through like an album, and the cursor follows. An arrow with no neighbor on its side does nothing, the picture stays. Any other key returns.

The companion floppy and disk swaps

Each CPU has its own companion: `A2FILECMD-6502-EXTRA.po` or `A2FILECMD-65C02-EXTRA.po` (also supplied as `.dsk`). Boot the matching `A2FILECMD-<CPU>-BOOT.po` first. Use images from the same release: native overlays retain their build-signature check. Each companion contains the 17 tools absent from BOOT, BASIC.SYSTEM, and copies of the menu and its catalog. No space is spent storing the other CPU's plugins.

With two Disk II drives, insert `/A2EXTRA6502` or `/A2EXTRA65C02` in **slot 6, drive 2**, matching the CPU of BOOT. The main disk's tools take precedence, and **!** lists tools from both disks. The current volume name is resolved on each access, so renaming the companion does not break loading.

With one drive, select the tool as usual. A missing file opens a message such as `Insert A2EXTRA6502 S6,D2: EDIT. 1/2 drive RET ESC`. Press **1** to choose **slot 6, drive 1**; the message then names that drive. Insert the requested disk and press **Return**. If the tool's input file is on the removed disk, A2FC next names that volume and waits for it before reading it. **Escape** cancels, including after a large overlay has loaded; the panels are restored. The last drive choice is kept for the session.

The catalog keeps all 41 commands selectable even when the companion is absent. The slot is currently fixed at 6; the drive can be 1 or 2. Disk swapping does not provide simultaneous access to two files on different floppies in one drive: tools whose source and destination must both be online still need a second drive, `/RAM` or a hard disk.

BASIC.SYSTEM is also searched for on the companion. With one drive, keep the Applesoft program on `/RAM` or another online volume so BASIC can read it after launch. After running an Applesoft program, reinsert the main floppy if needed and use `-/A2FC6502/A2FILE.SYSTEM` or `-/A2FC65C02/A2FILE.SYSTEM` to return to the matching BOOT floppy (adjust the volume name if renamed).

More tools in the ! menu

The complete edition includes these fifteen service-table overlays. The six marked **both** also ship on the floppy. Select the file, directory or volume first, then press **!** and choose the tool. A letter selects the next matching initial; Left/Right turns the menu page.

Tool	Edition	Operation
TXTCONV	both	Convert the selected text: C = CR, L = LF, D = CRLF, H = clear high bit, S = set it, T = expand tabs, A = transliterate UTF-8 accents. Choose in place or the other panel.
DATE	both	S enters twelve digits `DDMMYYYYHHMM` (1940–2039); F stamps modification dates on tagged files, or the selection. The date comes from ProDOS; a hardware clock can replace a manually entered value.

Tool	Edition	Operation
VERIFY	both	Read every block of a selected volume, or every byte of the selected file, and report read errors. ESC interrupts volume verification. This is a read test, without certification writes.
TAGPAT	both	Match = (any string) and ? (one character), with comma-separated filters: `T04` for TXT, `>2000` or `<2000` for size, `D` for modified today. T tags, U untags, X replaces the selection.
VOLINFO	both	Read-only ProDOS volume audit: free/used blocks, fragmented files, shared references, used blocks marked free, lost blocks and count mismatches. M opens an 80-column bitmap; N/P change page. ESC cancels a scan or returns.
VOLNAME	both	Rename a ProDOS volume and update the panel paths, including the program path when renaming its boot volume.
WIPE	both	F zeroes free blocks after Y/N confirmation; W zeroes the whole volume after `ERASE`. Whole-volume wiping refuses the running program's volume.
FIXTYPES	complete	Set type and auxtype from suffixes on tagged files or the selection, optionally removing the suffix. Image suffixes and `.SYSTEM` stay.
GOTO	complete	Nine favourite directories: A adds the current directory, D and a digit removes one, 1–9 jumps. Saved in `A2FILE/GOTO.CFG`.
FIND	complete	Search the volume for a name pattern, or prefix the query with `''` to search file contents (case-insensitive). Choose a result and press Return to jump to it.
CRC	complete	CRC-32 of the selected file or each tagged file, compatible with `zlib.crc32`.
IDENT	complete	Identify archives, images, Applesoft, AppleWorks and text conventions from file contents; text statistics describe the first 512 bytes.
MDVIEW	complete	Read Markdown and long text with wrapped lines, inverse headings, lists and code; page forward or backward.
RENAME	complete	Rename tagged files or the selection: prefix, suffix, extension replacement/removal, or numbering. Enter BAK with E to set `.BAK`; fragments start with a letter. Conflicts are counted as skipped.
IMGCONV	complete	Convert `.PO`/`.HDV`, `.DSK`/`.DO` and `.2MG` containers into the other panel, preserving the image's blocks.
BOOTBLK	complete	Copy ProDOS boot blocks from the boot volume to another ProDOS volume after confirmation.
UNDELETE	EXTRA / XL	Browse deleted entries in the active ProDOS directory. N skips, R recovers a validated candidate to the other panel, on a different online volume; existing names are refused. The source is unchanged.
DISKCMP	EXTRA / XL	V compares online ProDOS volumes; I compares the selected image with a named image in the other panel. Supports PO/HDV, DSK/DO and ProDOS-order 2MG. S compares two Disk II disks using one drive, with named slot/drive prompts.
MKIMAGE	EXTRA / XL	Create an empty formatted PO or 2MG in the active directory. Choose 140 KB, 800 KB, 2/4/8 MB or 32,767 blocks. Existing names are refused; cancelled or incomplete new images are removed.

Tool	Edition	Operation
RESCUE	EXTRA / XL	Recover a selected file (F) or active ProDOS volume (V) to another online volume, with up to 30 read attempts per 512-byte chunk. Missing chunks are zero-filled and listed in a companion LOG.
SYNC	EXTRA / XL	Recursively copy missing or newer files from the active directory to the other panel after direction confirmation. Destination-only files remain. Copies are read back before replacing older files.
TREE	EXTRA / XL	Paginated directory tree, file sizes and cumulative directory totals. Space advances a page; Escape exits.

UNDELETE recovers standard seedling, sapling and tree file candidates, including sparse blocks, only when their retained pointers and block count agree and all referenced blocks remain free. Index halves swapped by ProDOS DESTROY are normalized in memory. The deleted storage type is inferred from EOF; reused, inconsistent, ambiguous and unsupported entries are refused. An error during DESTROY can leave a deleted entry with partially processed indexes or bitmap; the deleted marker alone never authorizes recovery. No source directory, index or bitmap is rewritten. This is recovery to a new file on another volume, not an in-place directory repair. Inspect recovered data before relying on it. Deleted directories and resource forks are not restored.

RESCUE writes `BASE.REC` for a file. Disk output is raw ProDOS block order in `BASE.P01`, `BASE.P02`, etc., at most 16,000 blocks per part; concatenate the parts in numeric order to reconstruct a PO image. A single part can be renamed with the PO suffix. `BASE.LOG` identifies every zero-filled chunk and records completion or interruption. Partial output is retained after cancellation or an output failure. Sources and destinations must be on different online volumes.

SYNC compares ProDOS modification date and time (1940–2039), preserves file metadata and refuses overlapping directory trees. Equal-date and newer destination files are skipped. It reserves `A2FC.SYNC` for a temporary copy and `A2FC.BAK` for rollback; pre-existing files with those names are never overwritten. An installation failure restores the original name where possible; a remaining backup stays under `A2FC.BAK`. SYNC and TREE support paths shorter than 64 bytes and up to 16 directory levels; unreadable, deeper or unsupported resource-fork entries produce an incomplete/error result. They do not silently claim success.

DISKCMP reports differing block count and the first mismatch; read errors and cancellation do not produce an identical verdict. Single-drive comparison is exact and buffers two blocks per exchange, so it requires many disk changes; each prompt names the expected volume, slot and drive. Keys 1/2 change the drive. MKIMAGE creates a data volume without an operating system or boot program. The 32,767-block limit keeps a complete image within ProDOS's single-file size limit.

FIND keeps up to 20 matches and a bounded directory queue; it reports when its queue fills. DATE does not install a session clock driver or change creation dates or volume dates. VOLINFO audits the volume containing the active panel, or the selected volume in the volume list. It handles seedling, sapling, tree and extended files, and nested directories (up to 16 levels including the root). Sparse holes break a run; fragmentation means nonconsecutive data blocks within a fork. The bitmap shows one block per character: `.` free, # allocated. Large volumes need one traversal per 4,096 blocks, so XL scans take longer. Unreadable, unsupported or structurally invalid directories produce an incomplete result; lost blocks are reported only after a complete traversal. Counts saturate at 65,535. VOLINFO never writes to the volume. DOS 3.3, per-file block lists, report export and repairs remain planned.

VERIFY handles the selected file, without a tagged batch mode. These limits and the remaining disk tools are tracked in TODO.md.

Long operations: the bar, ESC, and what the panels show

A copy, a move or a delete of more than a moment shows its progress on line 22, across the whole line: the file counter (`3/12`), the name of the file in hand, a forty-cell bar and the bytes done over the size — for a delete, the items done over the count. The bar is only redrawn when a cell or the name changes, so it costs the copy nothing. **ESC** interrupts any of them at the end of the file in hand: a half-copied file is removed from the target, the panels are re-read, and the message says how far it got (`Interrupted: 2 of 5 done.`). Meanwhile the panels keep up: each file copied appears in the target panel as it lands, each file moved or deleted leaves its panel the moment it is gone — not only when everything is over. `bench/ops.py` reads the bar during a 300 KB copy, interrupts it, and watches the three-file copy and delete.

The mouse

An AppleMouse II card, in any slot, is recognized at startup (`Mouse` appears in the separator line) and the keyboard stays whole: nothing requires the mouse. The pointer, a MouseText arrow, appears only once the mouse is moved, and then follows the mouse over the panels. A click on a line selects it — the panel becomes active if it was not — and a second click on the selected line opens it, like RETURN. A click on a panel's column line changes the sort, a click on its path goes up to the parent directory. A click on a key-bar button is worth the key. In the viewers, the editor and the questions, only the keyboard answers. The card is read in passive mode, without interrupts: the Mockingboard keeps its own. In the text and hex viewers: **SPACE**, **RETURN** or **Down** for the next page, **B** or **Up** for the previous page, **ESC** to return to the panels. The text viewer shows 22 lines per page, cuts lines beyond 80 characters and remembers the page starts it has met, up to 96 pages. The hex viewer shows 320 bytes per page: address, sixteen bytes and their ASCII rendering.

Pictures

On opening, A2FC reads the first eight bytes and the size:

Content	Recognized format	Display
`DHRR 1 0 0 $40`	RLE-compressed DHGR (DHRR v1 stream), 16,384 bytes decompressed	DHGR
`HGRR 1 0 0 $20`	RLE-compressed HGR (HGRR v1 stream), 8,192 bytes	HGR
8,192 or 8,184 bytes	raw HGR page	HGR
16,384 bytes	raw DHGR page, AUX bank then MAIN (the order of A2FC's files)	DHGR

The RLE decoder is written in C and serves both streams; a run that straddles the border of the two banks is cut at the `$4000` crossing.

The loading is never seen. Writing into $2000–$3FFF is writing into the displayed page: as long as the decoder works, the screen stays on text — the panels intact at $400–$7FF — and the picture lights up only once complete. Without that, paging through a directory showed the previous picture being overwritten by the re-read entry table, then the new one painted band by band, AUX plane before MAIN plane. load_image also puts the memory routing back on the main bank before any read: the 80-column firmware leaves 80STORE armed, and with HIRES still active from a previous picture a raw HGR page would have gone to the auxiliary bank.

A DHGR picture destroys the contents of /RAM, so A2FC rebuilds it fresh. ProDOS's virtual disk lives in auxiliary RAM, and the auxiliary half of a DHGR page ($2000–$3FFF in the AUX bank) belongs to it: 18 blocks, measured on the bench, and that is exactly where the data of a file written to /RAM begins. This is the machine's constraint, not a flaw in A2FC — double hi-res and /RAM share the same bytes — but it left a half-wrong volume, whose next write returned garbage.

On leaving a DHGR picture, A2FC therefore asks /RAM to reformat itself: it recognizes its driver by its $FF00 address in DEVADR ($BF10), like the formatter, and sends it the FORMAT command, with the language card switched to bank 1 as that driver requires (ram_format, in a2fc_mli.s — about forty instructions; the driver rebuilds the volume directory itself, there is no structure to write, and the call returns bank 2 of the language card to A2FC, not the ROM). The volume comes back empty and consistent, 119 free blocks out of 127, and the message line says so: /RAM was rebuilt empty. You lose what it held — it was already lost — but nothing is wrong any more.

A **plain HGR** picture writes only to the main bank: it does not touch /RAM and triggers nothing. The bench bench/run.py checks both cases. In a directory, Left and Right move to the previous or next picture among the files that look like a picture (type FOT, or a BIN the size of a page, or a name ending in .RLE). The bench decodes both of the floppy's test cards with the same algorithm, in Python, and compares the graphics page byte by byte, the auxiliary bank included.

The text screen is never cleared. The panels stay at $400–$7FF throughout the paging, and A2FC rewrites only what changes there: the message line says Loading NAME... as soon as an arrow is pressed, even before re-reading the directory — the names of the two neighbors are kept out of the graphics page — then the cursor moves to the new picture (two lines rewritten, the whole panel only if it scrolls) and the info line follows. On return, both entry tables are re-read, but a panel is redrawn only if its re-read shows something other than at entry: /RAM rebuilt fresh under a panel, a floppy swapped. It is panel_hash (a2fc_mli.s) that judges, a fingerprint of the entry table folded into one word, far cheaper than a redrawn panel.

The loader and the decoder are an overlay. They are not in the resident program but in A2FILE/IMAGE.PLG, which A2FC reads into the $1B00–$1FFF window on the first picture and keeps in place until another overlay (TEXT.PLG, HEX.PLG, DELETE.PLG, HELP.PLG) replaces it; see "Building and memory". Without that file, or with one from another build, the message line says so and nothing shows.

Running an Applesoft program

A BAS file does not run on its own: it is BASIC.SYSTEM that runs it. **X** (or **RETURN**) on a BAS therefore loads BASIC.SYSTEM — from the root of the program's volume, its usual place, or failing that from the volume A2FC booted from, so a BAS on a data disk, on /RAM or on the hard disk runs with the floppy's BASIC.SYSTEM — and passes it the program name in the buffer that all its launchers use — Bitsy Bye included: the first eight bytes of a SYSTEM program are a jump then a name preceded by its length, at $2006, and BASIC.SYSTEM turns it into the –NAME command at startup. chain_command (chain.s) drops that name into the stub on page $0300, which writes it at chain_addr+6 just before jumping.

The program is launched by its full path (–/VOL/DIR/NAME) as long as it fits the 46 characters the stub has room for, so it resolves whatever the prefix is; BASIC.SYSTEM then sets the prefix to its own volume, which is why –A2FILE.SYSTEM brings A2FC back whenever BASIC.SYSTEM came from its disk. A longer path falls back to the old way: prefix on the program's directory and –NAME, with no return. Without any BASIC.SYSTEM — neither on the program's volume nor on the boot volume — the launch stops on Run failed: file not found and A2FC keeps control.

Coming back to A2 File Cmd. As with a SYS, A2FC does not take control back on its own. But from the Applesoft] prompt, –A2FILE.SYSTEM relaunches the launcher, which reloads A2 File Cmd — the program says so on screen. Two details make this return reliable, where an earlier version froze the machine on a blank screen just after switching to 80 columns:

- **The C stack.** The launcher and A2FC take over the whole machine: they read A2FILE.CODE up to $BE40, over BASIC.SYSTEM if it was there. But crt0, seeing BASIC.SYSTEM resident, placed the C stack on its HIMEM (~$9600) — right in the middle of the code being loaded, which the stack then overwrote as it grew. src/crt0.s (A2FC) and src/crt0_loader.s (the launcher) therefore always place the stack at $BF00, never on BASIC.SYSTEM's HIMEM, and always quit through the ProDOS dispatcher.

- **The prefix.** Everything A2FC loads — A2FILE/A2FILE.CODE, the overlays, the help, A2FILE.CFG, the formatter — is a path relative to the ProDOS prefix, which must therefore be the directory holding A2FILE.SYSTEM and A2FILE/. On a cold boot ProDOS sets it to the boot volume, and Bitsy Bye to the directory of the program it launches; the launcher keeps whatever it finds (src/loader_mli.s, GET_PREFIX), which is what lets A2FC live anywhere on a hard disk, not only at the root of a volume called /A2FILECMD. BASIC.SYSTEM, though, empties the prefix when it launches a SYS, but leaves the full path of that SYS at $0280: the launcher then takes its directory (/VOL/DIR/A2FILE.SYSTEM → /VOL/DIR/), falling back on ON_LINE on the last device $BF30 for a bare volume name. All of this before opening A2FILE.CODE, so that the relative path resolves. A2FC then reopens its panels and re-reads A2FILE.CFG, as on a cold boot. The FORMAT overlay returns directly and preserves the panels and prefix. bench/subdir.py also checks it from /HD/APPS.

Disk images

W opens the A2FILE/DISKIMG.PLG overlay, three ways to move a whole floppy, the one ground where A2Command still had the edge:

- **Write** an image (.PO ProDOS order, .DSK/.DO DOS 3.3 order, .2MG whose header gives the order) to a floppy. The selection is the image; you choose the target drive, which must be formatted.

- **Read** a floppy into a fresh image, dropped in the active panel's ProDOS directory, in ProDOS or DOS 3.3 order as you choose.

- **Copy** a floppy onto another. With a single drive, you choose the same one twice: A2FC reads one pass, asks for the target floppy, writes, asks for the source again, and so on.

The blocks go through READ_BLOCK and WRITE_BLOCK, whatever the driver (Disk II, SmartPort, /RAM). A pass's buffer is in main RAM ($3400, six blocks) and, for the single-drive copy, eighty more blocks in auxiliary RAM $2000: /RAM is therefore rebuilt fresh on exit, as after a DHGR picture. The order conversion is that of po2dsk.py. Like the formatter, writing requires the word ERASE in capitals, the program's own floppy is refused, and the warning names the drive and its volume; ESC cancels before any write. The bench reads the boot floppy into a DOS 3.3 image and compares it byte by byte, writes a 64-block image to a blank floppy and verifies the floppy after ejection, then carries the copy up to the warning.

A disk image as a folder

RETURN on a .PO, .2MG, .DSK or .DO file opens it read-only as a folder: the panel shows the contents of the image's ProDOS volume, and you navigate it as everywhere else — **RETURN** into a subdirectory, **ESC** to go up, **ESC** at the root to leave and find the image file back in its directory. The path carried by the panel becomes /VOL/DISK.PO/SUBDIR. A2Command cannot do this.

The blocks are read by fseek in the image file (img_read_block): a .DSK/.DO is in DOS 3.3 order, permuted sector by sector like po2dsk.py in reverse; a .2MG gives its order and the offset of its data in its header. The directory is then read block by block following the ProDOS chaining, exactly like a real directory. An image that is not a readable ProDOS volume (a DOS 3.3, for instance) is refused and the panel returns to its directory.

An image opened this way is **read-only**: only navigation, tagging (SPACE) and **C** act; the commands that would write are refused in plain text. **C** extracts the tagged files — otherwise the one under the cursor — to the other panel's ProDOS directory, with their type and their auxtype. Subdirectories are to be entered and extracted one by one; seedling and sapling files (≤ 128 KB) are extracted, the rare tree files are refused. The extraction is the `IMGFS.PLG` overlay: it reads the ProDOS file (a data block for a seedling, an index block of 256 pointers for a sapling, a null pointer being a hole of zeros) and writes it to the disk. The bench opens `TINY.PO`, descends into a subdirectory, extracts a file from it and compares its contents **byte by byte**.

A DOS 3.3 floppy

A2FC also reads **DOS 3.3** floppies, both from an image (`.DSK`/`.DO`, or a `.2MG` in DOS order) and from a **real disk** in a drive — that is what the first trials asked for. A `.DSK` image that is not a ProDOS volume is tried as DOS 3.3: if its VTOC (track 17, sector 0) is valid, its catalog opens as a flat folder. A real DOS 3.3 disk, which has no ProDOS volume, appears in the list of volumes (`/`) under the name `DOS 3.3`, with its slot and its drive: RETURN opens it. The DOS types (T, I, A, B) are shown as their nearest ProDOS type (TXT, INT, BAS, BIN), and **C** extracts the tagged files to the other panel's ProDOS directory, the DOS header removed (the two length bytes of an Applesoft or Integer, the four of a binary) so the file is usable.

The sectors are read by the same code, whatever the source: `fseek` in an image, or **READ_BLOCK** on the Disk II driver for a real disk, the logical DOS sector translated to a ProDOS half-block by the `DOS_TS` table, the inverse of the one in `po2dsk.py`. The chained catalog on track 17 and the sector-by-sector lists of the files do the rest. The catalog is read by the kernel, the extraction by the `DOS33.PLG` overlay. The bench opens a DOS 3.3 image, checks its types and extracts a file from it, compared **byte by byte**. (POM2 makes readable only the boot floppy and an image file; reading a real physical disk shares all the code of reading an image, only the source of the sectors changes.)

A raw DHGR page and the auxiliary bank

A 16 KB raw DHGR file is two planes: the first belongs in the auxiliary bank. A2FC reads it 512 bytes at a time into main memory and carries each chunk over with `AUXMOVE`, rather than setting `80STORE` and letting ProDOS write into the routed window — a trick that works on many machines but that ProDOS does not promise, its calls being specified for main memory only. Where it failed, a raw DHGR came back as `not an image` while the same picture in RLE went through, that path having always copied with the processor. The main plane is read straight in: it is ordinary memory.

VDrive: two volumes over the serial line

If a serial card answers at startup — a Super Serial Card, or a //c's built-in port, identified by the Pascal 1.1 signature of its ROM and a 6551 that responds; **slot 2 is probed first** (the //c's modem port, the usual place of a modem on a IIe; its printer port in slot 1 carries the same signature and the same 6551, and 0.6.7 took it by mistake), then 1, 3 to 7 — A2FC installs a **VDrive**: two ProDOS block devices, in the first slot whose drives 1 and 2 are free, served by whatever sits at the other end of the cable at 115 200 bps: the ADTPro server with its `Virtual.po` / `Virtual2.po`, `veserver.py` from ProDOS-Utils, or a Raspberry running Colin Leroy-Mira's `surl-server` — the same wire protocol that ADTPro's `VSDRIVE` speaks. The status line says so (`VDrive: serial card in slot 2, volumes in slot 1, drives 1 and 2.`), the volumes appear in the list like any disk, and copying to and from them is plain copying. The host also sends its date and time with every block read, and A2FC sets the ProDOS clock from it: a IIe without a clock card dates its files all the same.

The driver (`src/vsdrive.s`) lives *inside the program*, the way Ammonoid does it, not inside ProDOS: it is installed by `vsdrive_install` before the panels are read, and a cc65 destructor removes it — `DEVADR` restored, the two units taken out of `DEVLST`, DTR dropped — on Q, X, F and every other way out, so ProDOS never points at code that is gone. The main window being full, the driver and its 6551 layer sit in the language card (segment `LC`); ProDOS calls its drivers from *its* bank of that card, so a 17-byte thunk in page `$0300` (free under ProDOS; `chain.s` only borrows it after the destructor has run) switches bank 2 in, calls the driver, and puts bank 1 back read/write before returning to the MLI. The card being read-only at run time, the driver's variables live in page 3 as well. Each block is an envelope — `$C5`, the command (read 3/5, write 2/4 by drive), the block number, their XOR — then 512 bytes and their XOR; a read answer also carries four bytes of date and time. Interrupts are masked for the duration of a block (a byte comes every 87 cycles at that speed; the receive loop takes about 60), and every byte waited for has a 0.3 s timeout, so a host that is absent or unplugged gives a clean I/O error (`$27`) instead of a hang — the volume list then shows the drive without a name, as it does for an empty Disk II. Two things the bench taught: ProDOS's own buffer for `ON_LINE` and directory blocks (`GBUF`, `$DC00`) is in *its* bank of the language card, where a store from ours cannot land — the two accesses to the caller's buffer therefore go through page 3 as well, bank 1 for the time of one byte; and a 6551 raises an interrupt whenever DCD or DSR changes, whatever its registers say, so unplugging a cable that carries those lines would have killed ProDOS (`RESTART SYSTEM – $01`) — the driver registers a small ProDOS interrupt handler (page 3 too) that reads the status register, which acknowledges the interrupt, and claims it when bit 7 says it was ours.

`bench/vsdrive_server.py` is a host for the protocol over a TCP socket, and `bench/vdrive.py` the bench that plugs it into POM2's Super Serial Card bridge (`pom2_playtest --ssc PORT`, raw mode): the two volumes appear in the list, a file is read from the remote image and one is copied to it and read back on the host byte for byte, then the host is cut off and the volume list still comes back, and Q leaves `DEVLST` as it found it. It has not yet been tried on a real Super Serial Card or //c: tell me what yours makes of it.

Reading an AppleWorks document

An AppleWorks word-processor file (type `$1A`, shown as `AWP` in the Type column) opens with **RETURN** or **T** in a reader that works like the text viewer: 22 lines a page, SPACE or RETURN for the next page, B for the previous one, ESC to come back. The file is 300 bytes of header (byte 183, the minimum version, says whether two more bytes follow it — AppleWorks 3.0 files) then one record per line, two header bytes each: a carriage-return record (`$D0`), a formatting command (`$D1` and above, skipped), the end (`$FF`), or a text line whose first byte counts what follows. Inside a line the codes below `$20` are formatting — bold, underline, superscript, dates and page numbers — and are skipped; `$16` and `$17`, the tabs, are rendered to the next stop of 8. A ruler line (first byte `$FF`) is skipped. Every text record is one screen line, exactly as AppleWorks wrapped it.

`A2FILE/AWP.PLG` is a small overlay in C (`awp_entry` in `a2fc.c`), launched by the `!` menu as well. `tools/mkawp.py` writes and reads the format on the host — the `LETTER` of the `.2mg`'s `DEMO` comes from it — and `bench/awp.py` checks, line by line, that the page the Apple IIe shows is the one written.

Unpacking a ShrinkIt archive

`.SHK` is the NuFX archive of ShrinkIt, the way Apple II software has been packed and shared for decades. Put an archive under the cursor, press `!`, and pick **Extract a ShrinkIt .SHK archive**: every file in it is written into the directory shown in the *other* panel, its ProDOS type and auxtype restored, its name shortened to a valid ProDOS name. Compressed files are decoded on the way out — both the LZW/1 of the ProDOS-8 ShrinkIt and the LZW/2 of GS/ShrinkIt and CiderPress, as well as stored (uncompressed) files; the RLE pre-pass too. A disk image inside an archive comes out as a `.PO`, which you can then open as a folder (see above). Resource forks and comments are skipped.

The decoder lives in A2FILE/UNSHRINK.PLG. Its heart is hand-written assembly (src/unshrink.s): the 4 KB LZW dictionary does not fit beside the code in the main window, so it lives in auxiliary memory, and the decode loop is copied there and runs from it, reading and writing the aux bank around a RDAUX switch. That is also why the target cannot be /RAM: the ProDOS RAM disk shares that same aux memory, so it is rebuilt empty after each extraction, and the program refuses /RAM as a destination. The C driver (unshrink_entry in a2fc.c) walks the NuFX headers, opens the files and feeds the core block by block. tools/mkshk.py builds and reads the same format on the host (verified byte-for-byte against nulib2); bench/shk.py extracts a stored, an LZW/1 and an LZW/2 archive in the emulator and compares each result to the original.

Unpacking a Binary II archive

.BNY (or .BQY) is Binary II, the older and simpler wrapper that carried Apple II files over modems and BBSes: each file is preceded by a 128-byte header holding its name, type, auxtype and length, and padded to a 128-byte boundary. Put an archive under the cursor, press ! and pick **Extract a Binary II (.BNY) archive**: every file goes into the directory shown in the *other* panel, with its ProDOS name, type and auxtype restored. Binary II carries no compression of its own; a .BQY whose members are themselves ShrinkIt-packed comes out as .SHK files, which the ShrinkIt extractor above then unpacks. Directory entries inside an archive are skipped.

A2FILE/BINARY2.PLG is a small overlay written entirely in C (binary2_entry in a2fc.c); tools/mkbny.py writes and reads the format on the host (verified against nulib2) and bench/bny.py extracts a two-file archive in the emulator and checks each name, size, type and auxtype.

Formatting a disk

F or **FORMAT** in the **!** menu loads A2FILE/FORMAT.PLG, a native large overlay on BOOT and XL for each CPU. Escape returns directly to the panels, without relaunching A2FC or depending on saved settings. The overlay lists slot, drive, device type, current volume and capacity. The program's volume is marked **IN USE** and refused, including when A2FC is installed in a subdirectory. Choose a drive, enter a volume name (BLANK by default), then type **ERASE** in capitals and press **Return**. Escape or any other word cancels; nothing is written before confirmation.

Physical Disk II formatting also clears /RAM. Its track buffer needs auxiliary memory to preserve the resident program. The confirmation screen states this: copy any files in /RAM elsewhere first. Physical formatting is refused when A2FC itself runs from the RAM disk. This extra RAM reset is not needed when formatting a SmartPort or other block device. Music stops when opening FORMAT.

A Disk II floppy is physically formatted, track by track with the progress on screen: it is Jerry Hewett's (1985, public domain) and Gary Desrochers's (1989) ProDOS Hyper-FORMAT as ADTPro took it up, split into three calls to show the progress (format_diskii.s). The leading GAP1 is lengthened by 512 sync bytes so a written track covers a full revolution, whatever the drive or the emulator. A SmartPort that allows it receives the low-level format command, /RAM the one of its driver, a hard disk nothing. Then, for all of them, the ProDOS structures are written by WRITE_BLOCK (format.c): the Hyper-FORMAT boot at block 0, the root catalog at blocks 2 to 5 with the clock date, the allocation table starting at block 6. The boot and the header are read back and compared. bench/format.py checks both CPUs: cancellation, boot-volume protection, a write-protected floppy, physical formatting, /RAM, a 65535-block SmartPort volume, memory restoration, the C stack and direct return.

FORMAT code and strings stay below $3C00; state occupies $3C00–$3DFF and the MLI block buffer $3E00–$3FFF. While writing a physical track, format_diskii.s preserves resident $6500–$80FF in auxiliary RAM and restores it before returning to C, including on error. No resident call or interrupt can run while that region holds the track image. The buffer also covers the final sixteen bytes emitted by Build beyond the write loop's $8000 limit. The linker bounds the overlay and state separately.

The Mockingboard music

RETURN on a `.MB` file (MB1 stream, 2,304 bytes at most) loads it into auxiliary memory through the six-voice player and plays it once, on interrupt: the navigation, the viewers and the editor continue during the music, and disk reads do not stop it (verified in the emulator: the stream cursor advances during directory reads and picture loading). On a 5.25" floppy, ProDOS's Disk II driver disables interrupts during each block read: the player freezes for the read, then resumes; A2FC can do nothing about it. P pauses and resumes it, another `.MB` replaces it, Q and X cut it; once the piece is finished, P says so. The card is looked for in slots 1 to 7 at the first request; without a card, A2FC says so.

A loaded piece rebuilds /RAM fresh, like a DHGR picture. The stream lives at `$1000`–`$18FF` of the auxiliary bank, and this memory belongs to ProDOS's virtual disk: measured in the emulator with ProDOS 2.4.3, its driver puts blocks 9, 26, 43… (one in seventeen) there, its block map is at `$0C00` and its directory, a single block, at `$0E00`; nowhere in AUX are there 2,304 bytes out of its reach. Before the 2026-09-07 fix, the 56-byte fanfare was enough to overwrite block 9 of `/RAM`, silently. Now A2FC loads the stream, asks `/RAM` to reformat itself (`ram_format`, as on return from a DHGR picture) and writes it in the message line: `Playing WELCOME.MB, slot 4. P pauses.  /RAM was rebuilt empty.` The rebuilt volume never re-reads its free blocks, so the music plays without risk; writing to `/RAM` while it plays would spoil the piece, not the volume. Rebuilding `/RAM` also protects the small mirror that the player copies into the auxiliary bank at its code's address (above `$4000`, to read the stream under interrupt): on a `/RAM` filled up to there, this mirror would have been overwritten and the IRQ would have gone off the rails.

The text editor

`E` opens the file in a full-screen editor of 22 lines: arrows, DELETE to erase to the left, Ctrl-D to the right, Ctrl-A and Ctrl-E start and end of line, Ctrl-P and Ctrl-N previous and next page, Ctrl-T and Ctrl-B start and end of the text, RETURN splits the line, Tab inserts four spaces. The bottom bar gives the path, the line, the column, the size and a star if the text has changed. ESC opens the menu: S save, X save and exit, Q quit without saving (with confirmation if the text has changed), ESC continue. The file keeps its type and its auxtype; the text fits in the graphics page, so 8 KB at most, CR line endings, bit 7 stripped on loading. Lines longer than the screen are not wrapped.

What A2FC does not do

- It refuses to copy a directory into itself, and a tree in which one path adds up to more than 213 entries (the reserve for the recursive walks, housed in the inactive panel's table during the operation).

- It does not come back from a launched program: what is loaded overwrites it. FORMAT is an overlay and returns directly; it does not launch another program.

- A directory of more than 139 entries is read in windows, in disk order and without sorting: the header shows the rank of the first entry followed by a plus sign, and the cursor moves from one window to the next by crossing the edges. No directory on the volume reaches this size; this mode is not covered by the bench.

- It modifies no file on its own: only the C, V, R, D and K commands write to the disk.

Files on the disk

ProDOS name	Function
`A2FILE.SYSTEM`	The launcher, at the root: the only `.SYSTEM` file on the volume, hence the one ProDOS starts (`src/loader.c`)

ProDOS name	Function
A2FILE/A2FILE.CODE	The manager itself (src/a2fc.c, plus src/a2fc_mli.s for GET_FILE_INFO, SET_FILE_INFO and the /RAM reformat)
A2FILE/A2FILE.CFG	Written by A2FC on quit: the two directories, the sort and the active panel, three lines of text
A2FILE/FORMAT.PLG	Native formatter overlay (format.c, format_diskii.s, format_mli.s), loaded by F or the ! menu
A2FILE/A2FILE.HELP	The text of the help page (data/A2FILE.HELP.TXT), one line per element: x,y,KEY,label, x,y,#TITLE for a section, x,y,~text for plain text. It goes through the graphics page, none of the help stays in memory

The names fit in ProDOS's fifteen characters.

The 5.25" floppy

make disk produces six volumes, grouped alphabetically by processor:

CPU	BOOT, 140 KB	EXTRA, 140 KB	XL, 32 MB
6502	A2FILECMD-6502-BOOT.po	A2FILECMD-6502-EXTRA.po	A2FILECMD-6502-XL.2mg
65C02	A2FILECMD-65C02-BOOT.po	A2FILECMD-65C02-EXTRA.po	A2FILECMD-65C02-XL.2mg

Both floppy types also have a .dsk copy in DOS sector order. BOOT is a bootable ProDOS volume with the file manager, disk tools and formatter. EXTRA is a non-bootable companion holding the remaining plugins and BASIC.SYSTEM. EXTRA currently keeps 57 free blocks (28.5 KB) on 6502 and 58 (29 KB) on 65C02 for future tools. XL is bootable and contains all 42 overlays, BASIC.SYSTEM, DEMO/ and IMGHGR/ on the same 65535-block disk; it needs no EXTRA floppy.

ProDOS volume names also identify the CPU and role: /A2FC6502, /A2EXTRA6502, /A2XL6502, and /A2FC65C02, /A2EXTRA65C02, /A2XL65C02. Use BOOT and EXTRA from the same CPU and release. make disk ARCH=6502 builds the three 6502 volumes; ARCH=enh builds the three 65C02 volumes. The latter support MouseText and an optional mouse; the 6502 versions use the keyboard and run on the original IIe. make benchfloppy ARCH=enh keeps a private build/A2FILECMD-full.po with native overlays for regression tests; it is not published.

The 6502 build — a 6502 without the 65C02 opcodes, no MouseText; 128 KB and an 80-column card are still required — needs **cc65 master** (the git tree after 2.19: make; make install PREFIX=~/opt/cc65-head, pointed to by CC65_HEAD), because only there does the apple2 target have the 80-column console in plain 6502 (machinetype, aux80col, videomode); in 2.19 the 80-column console exists only in apple2enh, compiled for the 65C02. The enhanced build keeps the machine's cc65 2.19 on purpose, so that its byte counts do not move. What the 6502 variant changes: stz/bra macros and (zp),y in music.s; the launcher's machine check accepts $FBC0 = $EA; BINARY2 becomes a big overlay; **no mouse** (A2FC_NOMOUSE: keyboard only), which is what pays for everything else — the resident window ends at $BC27, VDrive included. Master also changed a few runtime details that CC65_MASTER covers: callmain now defines _exit (ours replaces it), _oserror is ___oserror, &main is refused in C (the overlay link id is now a2fc_link_id, taken in assembly).

The continuous integration (`.github/workflows/ci.yml`) builds both on every push — cc65 2.19 from Ubuntu for the 65C02 build, cc65 master cloned at a pinned commit (`CC65_HEAD_COMMIT`) and cached for the 6502 one — checks the images, and a `v*` tag publishes the three of them with one `SHA256SUMS.txt` and notes taken from `CHANGELOG.md` (`tools/release_notes.py`) — the changelog is the only place a release's text is written.

One lesson the bench taught: the `apple2` target's `initostype` constructor calls the ROM (`$FE1F`) without switching it in, as a `.SYSTEM` launched by ProDOS may assume — but A2FC is launched by its own loader, whose `videomode()` leaves language-card bank 2 readable, so `$FE1F` landed in ProDOS and the machine died before `main()`. `crt0.s` now puts the ROM back (`bit $C082`) before running the constructors, for both builds. The benches run on the 6502 images with `A2FC_IMG=A2FILECMD-6502-BOOT A2FC_BUILD=build-6502`, and with `A2FC_PRESET=iie_unenh` they run on POM2's unenhanced IIe of 1983 (NMOS 6502, the 16 KB firmware without MouseText, `$FBC0 = $EA`) — the only machine that proves this build, since a `stz` or a `bra` there is an undefined opcode. The whole suite passes on it, and the 65C02 build shows its refusal there, as the real machine would.

`.po` and `.dsk`: one floppy, two layouts

The two floppy images hold the same ProDOS volume, byte for byte — the same `A2FILE/` subdirectory, the same files; **there is no DOS 3.3 in the .dsk**. What differs is only the order in which the sixteen 256-byte sectors of each track are laid out in the file:

- `.po` ("ProDOS order") stores the 512 bytes of ProDOS block *n* in one piece, block after block;
- `.dsk` ("DOS order") stores each track's sectors in the physical order DOS 3.3 gave them, so block 0 lands on track 0, sectors 0 and 14, block 1 on sectors 13 and 12, and so on down to block 7 on sectors 1 and 15. That is the layout `.dsk` files have had since the DOS 3.3 days, hence the name — but any content can be stored that way, DOS 3.3 or ProDOS.

Both files are 143 360 bytes, and once written to a real disk there is no difference at all. `po2dsk.py` does that permutation and nothing else. Two files are published because writing tools and emulators guess the layout from the extension: hand one a `.po` renamed `.dsk`, or the reverse, and the floppy it writes is unreadable. Take the one your tool expects; ADTPro, CiderPress and most emulators take the `.po`.

File	Content
`PRODOS`, `BASIC.SYSTEM`	ProDOS 8 2.4.3, the last stable version, and its Applesoft interpreter: freely distributed for the Apple II community, they are not the author's
`A2FILE.SYSTEM`	the launcher, the only `.SYSTEM` program: the floppy boots straight into A2FC. Compiled with `NO_CHDIR`, it relies on the ProDOS prefix — the directory it lives in — and rebuilds it only when `BASIC.SYSTEM` has emptied it
`A2FILE/A2FILE.CODE`, `A2FILE/*.PLG`, `A2FILE/A2FILE.HELP`	the program (with the VDrive serial driver), its overlays (`IMAGE`, `TEXT`, `HEX`, `HELP`, `DELETE`, `MUSIC`, `RUN`, `ATTR`, `IMGFS`, `DOS33`, and the big ones `EDIT`, `MENU`, `DISKIMG`: BINs loaded at `$1B00` on demand), the help text and the formatter; `A2FILE.CFG` will be written alongside
`IMGHGR/` (`.2mg` only)	nine raw HGR pages from POM1 (`data/IMGHGR`, ProDOS names in English: `ALIEN`, `BEAR`, `DRAGON`, `GOBLIN`, `LIZARD`, `MAZE3D`, `TIGER`, `UBERNIE`, `VILLAGE`): open one, then Left and Right leaf through the folder
`DEMO/` (`.2mg` only)	one example of everything A2FC can open, entirely computed by `tools/mkdemo.py`: the two test cards raw (`DHGR.RAW`, `HGR.RAW`) and RLE (`DHGR.RLE`, `HGR.RLE`), a three-voice fanfare (`WELCOME.MB`), a text (`SAMPLE`), an Applesoft program (`HELLO`), an AppleWorks document (`LETTER`), a ProDOS disk image as `.PO` and as `.2MG` (`TINY.PO`, `TINY.2MG`), a DOS 3.3 disk (`DOS33.DSK`), the text and the program packed by ShrinkIt (`SAMPLE.SHK`) and by Binary II (`SAMPLE.BNY`), and a `README` that says what to press

The floppy keeps 19 free blocks. At startup, the left panel shows the boot volume's root and the right panel its DEMO directory when there is one, the list of volumes otherwise. `mkvolume.py` writes both volumes (files above 128 KB, such as the 143 KB DOS 3.3 image, become ProDOS *tree* files), `po2dsk.py` derives the DOS 3.3 order and `po22mg.py` puts the 64-byte 2IMG header on the hard disk. The bench boots the floppy in slot 6 in POM2 with a scratch hard disk that carries its own DEMO; `bench/hd.py` boots the `.2mg` itself, in slot 7. Q hands control back to Bitsy Bye on the boot volume. A 2.5 alpha 8 version of ProDOS exists; it is not used, for lack of being published.

Building and memory

`make` builds the three binaries, `make disk` the floppy. The code runs at `$4000`. The two tables of 140 entries occupy the MAIN graphics page `$2000–$3FFF`, free as long as no picture is displayed: a picture covers it, and both panels are re-read on return, tags preserved. Low RAM `$1000–$1AFF` receives all of `a2fc.c`'s BSS (panels, paths, copy, text page starts, the mouse), zeroed by `main`, and since `a2fc.cfg` cc65's main BSS with it. Above, `$1B00–$1FFF` is **the overlay window**: 1,280 bytes where A2FC loads, on opening a file, the module that knows how to read it, and forgets it on returning to the panels. **Thirteen overlays**, under `A2FILE/`: `IMAGE.PLG` (the picture decoder), `TEXT.PLG` and `HEX.PLG` (the viewers), `HELP.PLG` (the help), `DELETE.PLG` (the D command and `delete_tree`, which moving a directory also loads), `MUSIC.PLG` (the loading of a `.MB`), `RUN.PLG` (X, F, and RETURN on a SYS or a BAS), `ATTR.PLG` (R, K, A, L), `IMGFS.PLG` (the extraction of a ProDOS image opened as a folder) and `DOS33.PLG` (the extraction of a DOS 3.3 floppy, see "A disk image as a folder"); `MUSIC.PLG` also carries M and S; and **three big overlays** that also take the graphics page `$2000–$3FFF`: `EDIT.PLG` (the editor, its text above its code), `MENU.PLG` (the `!` menu) and `DISKIMG.PLG` (the disk images). They are **linked with the program** — they call `fopen`, `memcpy`, `view_getc` like any function, and the kernel calls their entry point at its address in the window — but `a2fc.cfg` writes them to separate files (two segments per overlay, `NAME` for the code and `NAMER0` for the strings, in the same file `%0.NAME`, code first so that the zero words cc65 places at the head of a `.proc` under `-Cl` do not land on the entry point) that `make disk` puts under `A2FILE/` with the auxtype `$1B00`. Moving a module to an overlay handed **nearly 3.5 KB** back to the main window, which now ends at `$ADC2` instead of `$BADF`.

Each overlay opens on the `struct Overlay` header (`a2fc_plugin.h`, `overlay.s`, the first object of the link): the **signature** — the address of `main` in the link that produced it, or `PLUGIN_MAGIC` for a third-party overlay —, a **flags** byte (`OVERLAY_BIG`: big overlay), the **entry point** that the `!` menu calls on the selection, and a one-line **description**. An overlay from another build is refused like a missing one, the message line says so. **A2FILE.CODE and its .PLG therefore go together.** A third-party overlay, for its part, touches the program only through the **service table** (`struct A2fcApi`) that the kernel passes it: `fopen`, `message`, `confirm`, `prompt`, `dir_open`/`dir_next`, `read_panel`, `mli`... — the stable ABI, whose fields do not move and to which one only adds at the end (`api->version` says so). `overlay()` reads each overlay with a plain `fread`, and re-reads none as long as it is in place: paging through a directory of pictures re-reads nothing. The launcher now stages at `$1000` only the 3 KB of the language card image (`STAGE_BYTES` in `loader.c`). The `LOWRAM` region is bounded to `$0B00` so the linker refuses a BSS that would climb into the window, and `check_layout.py` verifies that each overlay fits between low RAM and its ceiling — the graphics page, or what a big overlay keeps there for itself — and that each file has the length of the link. The reserve for the recursive walks borrows the inactive panel's entry table.

The mouse (`mouse.s`) looks for the AppleMouse II card by its signature (`$Cn05=$38`, `$Cn07=$18`, `$Cn0B=$01`, `$Cn0C=$20`, `$CnFB=$D6`) from `$C7` to `$C1`, skipping slot 3, where the //e's 80-column firmware answers; `INITMOUSE`, then `CLAMPMOUSE` to 0..79 and 0..23 — the position arrives directly in screen cells, no calculation — and `SETMOUSE` mode 1: the card counts by itself, `READMOUSE` on each turn of `wait_key` is enough, no interrupt is reserved. The pointer is a MouseText character (`$42`) placed on the cell, the hidden byte being restored before any redraw; the even column lives in the AUX bank, reached by `PAGE2` under `80STORE`. The bench puts an AppleMouse II (HLE AppleWin) in slot 4 for the whole session and checks the pointer, the firmware bounds, the click, the second click, the panel switch and the key bar. The viewers, the inputs and the preferences file live in the language card, `$D400–$DFFF` in bank 2 (about twenty bytes free: `check_layout.py` watches), copied by `crt0.s` as for the game; before launching a program, A2FC puts the ROM back in for reading.

The ceiling of the main window. What survives initialization — CODE, RODATA, DATA, INIT — must end below the floor of the C stack, `__HIMEM__ - __STACKSIZE__`; only ONCE is allowed to exceed it, it is dead before `main`. ld65 does not check this: the BSS region is sized by `__HIMEM__ - __STACKSIZE__ - __ONCE_RUN__` and, as soon as this difference goes negative, it reads it as a whole unsigned number, reports nothing and places the BSS in the middle of the stack. The link succeeds, the program corrupts itself in use. Two measures close this trap: A2FC is linked by `a2fc.cfg`, where the BSS descends into low RAM (it links neither `malloc` nor `free` — the ProDOS buffers come from `$0800` — so no heap follows it), and `check_layout.py` checks the floor at each link. The C stack is 256 bytes: the bench measured its deepest low at **94 bytes** below `$BF00`, the recursive copy of a tree included, `-Cl` making the locals static. The 512 bytes before, plus the 88 of the BSS, are handed back to the code — enough to house the Applesoft launcher, where only about twenty bytes were left. The programs launched by X are launched by a stub copied to page `$0300` (`chain.s`), which reads the whole file at its address and jumps to it: no size limit. `chain_command` adds to it the name that `BASIC.SYSTEM` expects at `$2006`, 47 more bytes in the stub, and an assembly assertion keeps the whole thing below `$03D0`, where the vectors begin. A2FC no longer uses either `opendir` or `malloc`: directories are read like files, block by block, in the copy buffer, which is also faster. To house the editor and the viewer, it also gave back `hgr_loader.s` (the C decoder replaces it, in `memset`/`memcpy` slices up to the banks' border), `qsort` (an insertion sort) and `exec()` (a launcher of a few lines); the repeated messages are shared and the help lives on the disk. The two files open during a copy use the ProDOS buffers `$0800` and `$0C00`; A2FC does not use MAPBSS. The program is compiled with `-Cl` (static local variables); the three recursive walks (count, copy, delete of a tree) put their variables back on the stack with `#pragma static-locals`, without which the inner level overwrites the outer level's path length. The exit follows the ProDOS QUIT of the cc65 startup.

The proofs come from windowless POM2 benches, which all start from `dist/A2FILECMD-6502-BOOT.po` **as it will be downloaded**. `bench/run.py` plays a complete session — boot, navigation, pages, tagging, copy, move, rename, a lock refusing deletion, change of type and auxtype, directory creation, deletion, text and hex viewers, help, both test cards compared **byte by byte in both banks**, `/RAM` rebuilt after a DHGR and intact after an HGR, the fanfare that ends on its own, the editor, the formatter that lists the drives and relaunches the manager, the disk images written and read back (`.PO` and `.DSK`) and a floppy copied drive to drive, an image opened as a folder and a file extracted and compared, a DOS 3.3 catalog read and a file extracted, and finally an Applesoft program launched by `BASIC.SYSTEM` then the return to the panels by `-A2FILE.SYSTEM`: **69 checks**. `bench/memory.py` measures the low of the C stack by making the program work (86 bytes out of the 256 reserved), and `bench/smoke.py` merely checks that the published floppy boots. Off the emulator, `make test` checks the memory-layout contract, the round-trip of the ProDOS volume writer and the decoding of the demo files. The bench always works on a copy of the image.

Bugs fixed by this hunt (and an independent review of the code) before 1.0: the tag map was one byte too short (four phantom entries in a full window), the panel truncated long paths from the end, a window after the first counted 140 entries and repeated one, a directory emptied in windowed mode stayed stuck, the formatter computed the block map on 16 bits (null for 65,535 blocks), took the size from the old volume's header, accepted a drive with no disk, and returned to Bitsy Bye from the floppy (A2FILE.SYSTEM is at its root); a BIN loaded at `$0800` overwrote the stub's buffer; a damaged A2FILE.HELP or A2FILE.CFG could make it write anywhere; the copy destroyed the target before having opened the source, and could replace an empty directory with a file. The worst, found by the final review and reproduced by `roundtrips.py`: the launch stub did not return to ProDOS the interrupt entry taken at startup for the Mockingboard (ProDOS has only four, and the vector pointed into overwritten memory); on the third F/ESC round-trip, A2FC crashed into the monitor. `chain.s` now calls `donelib` (the cc65 destructors) before jumping, and the launcher does the same. A third review, centered on the editor, the pictures and the music, fixed still more: the editor had no visible cursor (`cursor(1)` from conio), a file created by E shifted the restored tags onto other entries (they are cleared in that case), a save on a full volume emptied the file before failing (the space is checked before the `fopen "wb"` that truncates), RETURN in the middle of a line left the old end on screen, the editor's open errors vanished under the redraw, the launcher read A2FILE.CODE without a bound below `$BF00`, the music's destructor came after the interrupt was freed (priority 11 in `music.s`), the picture viewer kept a stale index if the directory changed under it, and a .MB without END made it read AUX beyond the stream. `explore2.py` covers the editor (shortening, no final CR, LF only, locked file), the tags in a full window and in a second window, the move to /RAM and the deletion of a tree of 150 files.

The hunt of the evening of 2026-09-07, after the overlays and the mouse, found and reproduced in POM2 four more bugs: E on a file of more than 8 KB read it into the graphics page before refusing it, and left both entry tables overwritten by its start without re-reading them (the next panel line showed noise) — the size is now refused before any `fread`; a BAS launched without `BASIC.SYSTEM` on the volume (`Run failed`) left its name in `chain.s`'s stub, and the next SYS launched by X received that "-NAME" at `$2006`, in the middle of its code (blank screen, machine crashed) — the name is cleared as soon as the launch fails; a DHGR load that failed halfway (truncated file) had already written to the auxiliary bank but did not rebuild `/RAM` — it is rebuilt as soon as a DHGR load has begun; and the music, see above, overwrote `/RAM` silently.